



TÉCNICAS DE PROGRAMAÇÃO II

PROFº LUIZ CLÁUDIO

INTRODUÇÃO A ORIENTAÇÃO A OBJETOS

- **O que é um paradigma de programação?**

Um paradigma é a abordagem utilizada para resolver problemas por meio da programação de computadores. Existem vários paradigmas, incluindo o estruturado e orientado a objetos.

- **O que é o paradigma de POO?**

A POO é um modelo de análise, projeto e programação que se baseia na aproximação entre o mundo real e o mundo virtual. Ela envolve a criação e interação entre objetos, atributos, códigos e métodos.



INTRODUÇÃO A ORIENTAÇÃO A OBJETOS

- **Quais são os 4 Pilares básicos da POO?**
- **Encapsulamento:** Oculta detalhes internos de um objeto, permitindo que ele seja usado sem se conhecer sua implementação.
- **Herança:** Permite criar novas classes a partir de classes existentes, reutilizando código e estabelecendo hierarquias.
- **Polimorfismo:** Permite que objetos de diferentes classes sejam tratados de forma uniforme, facilitando a flexibilidade e extensibilidade do código.
- **Abstração:** Representa objetos do mundo real de forma simplificada, focando nos aspectos relevantes.



INTRODUÇÃO A ORIENTAÇÃO A OBJETOS

- **Quais são as vantagens de usar a POO?**
- **Reusabilidade:** Código pode ser reutilizado em diferentes partes do programa.
- **Manutenção:** Mudança de uma classe não afetam outras partes do código.
- **Organização:** Classes e objetos tornam o código mais organizado e modular.
- **Que linguagens de programação utilizam POO?**
- A maioria das linguagens modernas, como Java, Python, C#, C++, são baseadas em POO.



O QUE É UM OBJETO?



Um objeto é tudo aquilo que existe. Existem objetos que são criados em muitas aplicações ,poderia ser um cliente, um carro, um relógio. Em termo computacional um objeto é a representação de algo que existe.Você deve abstrair um "objeto" real para representá-lo computacionalmente, como, por exemplo, construir o objeto CARRO,RADIO.

Objetos são instâncias de classes;



ABSTRAÇÃO



Os carros apresentam diferentes formatos, cores e estilos. Cada objeto-carro tem característica semelhantes entre si, por exemplo todos tem quatro rodas, e no mínimo, duas portas, além de luzes de farol e de freio, bem como vidros frontais e laterais, porém cada carro possui sua característica própria.

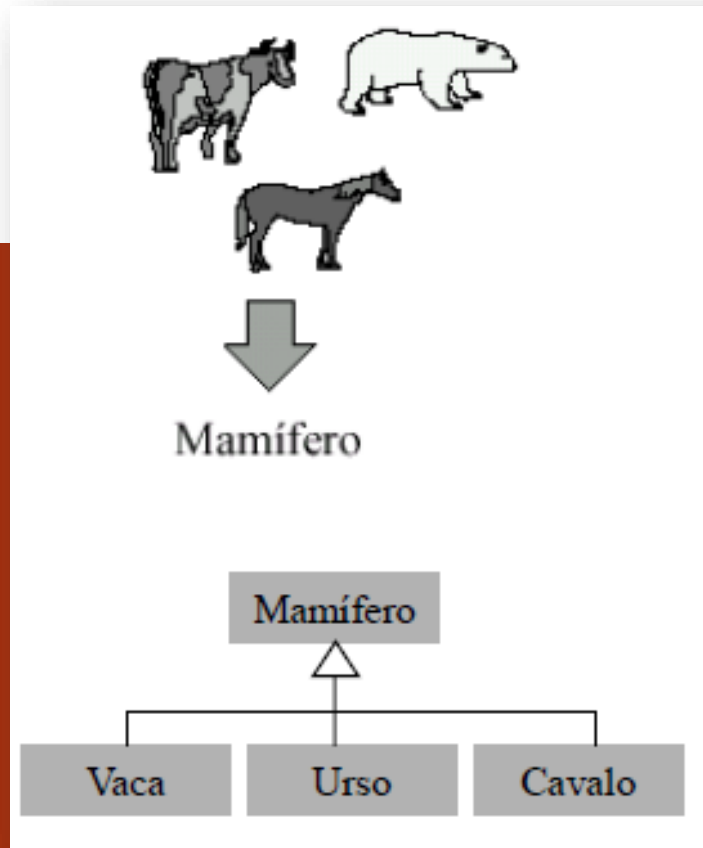
No momento compreendemos este conceito, percebemos que “fusca” é a denominação de um grupo, ou seja, é a abstração de uma classe: a classe carro.



ABSTRAÇÃO

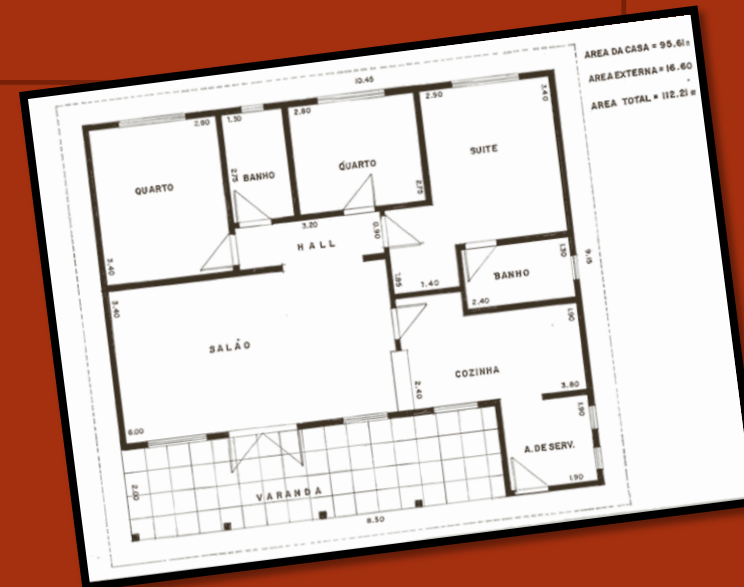
O que é abstração?

Podemos dizer abstração é generalização.



CLASSE

A criação de uma classe é feita em tempo de projeto, quando você está codificando o seu software, o que envolve a escrita do código fonte. O objeto é criado em tempo de execução, quando é feita uma instancia da classe é criado um objeto. A classe é como se fosse um molde, você constrói o molde e depois utiliza para fazer diversos objetos.



O QUE É UMA CLASSE?

■ Classe

- Uma estrutura de dados que contém métodos e atributos.

Carro
- marca : String - com : String - marcha : int - velocidade : int
+ movimentar (aceleração : int) : void + parar() : void + consultar() : int

- **Atributos:** descrevem as propriedades do objeto.
- **Métodos:** operações que descrevem ou modificam o comportamento do objeto.



CLASSE X OBJETO

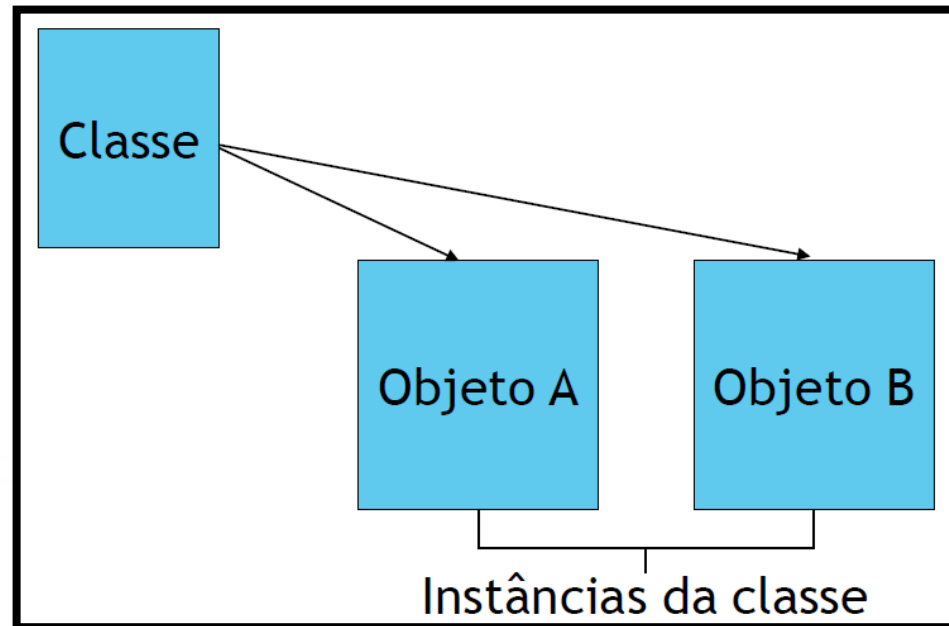


Objetos são instâncias de classes, que determinam qual informação conterá e como poderá manipulá-la. Um programa desenvolvido com uma linguagem de programação orientada a objetos manipula estruturas de dados através dos objetos, da mesma forma que um programa em linguagem tradicional utiliza variáveis.



CLASSE X OBJETO

- Classe representa um molde para os objetos.



EXEMPLO CRIAÇÃO DE CLASSE

```
class Pessoa:
    def __init__(self, nome, idade, ativo):
        self.nome = nome
        self.idade = idade
        self.__ativo = ativo

    def ativar(self):
        self.__ativo = True
        print("A pessoa foi desativada com sucesso")
```

Nome da Classe
(primeira letra
maiúscula)

Método
construtor e
seus parâmetros

Atributos
recebem valores
dos parâmetros
do método
construtor

Método ativar()



EXEMPLO COM CLASSE CARRO

```
class Carro:
    #construtor
    def __init__(self,nome):
        #atributo
        self.nome = nome

    #método
    def acelerar(self):
        print(f"{self.nome} está acelerando...")

#define objeto car e define parâmetro como Fusca
car = Carro('Fusca')
#mostra mensagem passando o parâmetro nome
print(car.nome)
#chama o método acelerar da classe Carro
car.acelerar()

#define objeto c e define parâmetro como Celta
c = Carro(nome = 'Celta')
#mostra mensagem passando o parâmetro nome
print(c.nome)
#chama o método acelerar da classe Carro
c.acelerar()
```



EXEMPLO COM CLASSE PESSOA

```
class Pessoa:

    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

    def calculaIdade(self):
        ano = int(input("Digite o ano atual:
"))
        return ano - self.idade

p1= Pessoa('Joao', 35)
p2 = Pessoa('Maria', 23)
print(p1.calculaIdade())
print(p2.calculaIdade())
```



ENCAPSULAMENTO

- Encapsulamento ou “*data hiding*” é um conceito bastante importante em orientação a objetos.
- É utilizado para restringir o acesso às declarações de uma classe.



ENCAPSULAMENTO

MODIFICADORES DE VISIBILIDADE

- ► **public:** A classe, método ou atributo, são sempre acessíveis em todos os métodos de todas as classes.
- ► **private:** A classe, método ou atributo, são acessíveis somente por métodos da própria classe. É o nível mais rígido de encapsulamento.
- ► **protected:** A classe, método ou atributo, são acessíveis nos métodos da própria classe e suas subclasses (herança).



ENCAPSULAMENTO

DEFINIÇÃO DO ATRIBUTO PRIVATE

```
class Pessoa:  
    def __init__(self, nome, idade):  
        self.__nome = nome  
        self.idade = idade
```

Atributo private(privado)

O que define o atributo privado são dois underlines



ENCAPSULAMENTO

GETTERS E SETTERS

- **Getters e Setters**
- ► No Python, há uma convenção de nomenclatura para os métodos que têm como finalidade acessar ou alterar as propriedades de um objeto.
- ► Segundo essa convenção, os nomes dos métodos que permitem a consulta das propriedades de um objeto devem possuir o prefixo **get**. Analogamente, os nomes dos métodos que permitem a alteração das propriedades de um objeto devem possuir o prefixo **set**.



ENCAPSULAMENTO GETTERS E SETTERS

- ► ***set_Atributo(self, atributo)***: método utilizado para alterar o conteúdo de um atributo de uma classe, encapsulado com *private*;
- ► ***get_Atributo(self)***: método utilizado para acessar o conteúdo de um atributo de uma classe, encapsulado com *private*;



ENCAPSULAMENTO

EXEMPLO DE ENCAPSULAMENTO CLASSE PESSOA

```
class Pessoa:  
    def __init__(self, nome, idade):  
        self.__nome = nome  
        self.idade = idade
```

```
    def get_nome(self):  
        return self.__nome
```

```
    def set_nome(self, nome):  
        self.__nome = nome
```

Atributo nome com Getter e Setter



ENCAPSULAMENTO

EXEMPLO DE ENCAPSULAMENTO CLASSE PRODUTO

```
#Define a classe Produto
class Produto:
    #define o construtor self      construtor= utilizado para inicializar um objeto logo após a criação
    def __init__(self):
        self.__nome = ""
        self.__valor = 0
        self.__quantidade = 0

    # getter nome
    def get_Nome(self):
        return self.__nome

    # setter nome
    def set_Nome(self, nome):
        self.__nome = nome

    def get_Valor(self):
        return self.__valor

    def set_Valor(self, valor):
        self.__valor = valor

    def get_Quantidade(self):
        return self.__quantidade

    def set_Quantidade(self, quantidade):
        self.__quantidade = quantidade
```

Definição dos atributos

Encapsulamento dos atributos , getter e setters



ENCAPSULAMENTO

EXEMPLO DE ENCAPSULAMENTO CLASSE PRODUTO

Método cadastrar

```
def cadastrarProduto(self):  
    print("\n=== Cadastro de Produtos ===")  
  
    nome = input("Digite o nome do produto: ")  
    quantidade = int(input("Digite a quantidade do produto: "))  
    valor = float(input("Digite o valor do produto: "))  
  
    self.set_Nome(nome)  
    self.set_Valor(valor)  
    self.set_Quantidade(quantidade)  
  
    print("Produto cadastrado com sucesso!")
```

Para armazenar os valores digitados dentro dos atributos são utilizados os setters ex: setNome

Método mostrar

```
def mostrarProduto(self):  
    print("\n=== Dados do Produto ===")  
    print("Nome do Produto:", self.get_Nome())  
    print("Quantidade:", self.get_Quantidade())  
    print("Valor:", self.get_Valor())
```

Para mostrar os valores usa os getters ex: get_Nome

#método calcular

```
def valor_total(self):  
    return self.__valor * self.__quantidade
```



ENCAPSULAMENTO UTILIZANDO PROPRIEDADES

Getters e Setters: usando *Properties*

```
class Pessoa:
    def __init__(self, nome, idade):
        self.__nome = nome
        self.idade = idade
```

```
@property # get
def nome(self): return
    self.__nome
```

```
@nome.setter #set
def nome(self, nome):
    self.__nome = nome
```

Ao utilizar properties , o encapsulamento dos métodos é visto como se fosse um método



ENCAPSULAMENTO

EXEMPLO PROPRIEDADES CLASSE FORNECEDORES

```
class Fornecedores:

    def __init__(self):
        self.__nomeFornecedor = ""
        self.__nomeProduto = ""
        self.__descricaoProduto = ""

    # PROPERTY nomeFornecedor
    @property
    def nomeFornecedor(self):
        return self.__nomeFornecedor

    @nomeFornecedor.setter
    def nomeFornecedor(self, valor):
        self.__nomeFornecedor = valor

    # PROPERTY nomeProduto
    @property
    def nomeProduto(self):
        return self.__nomeProduto

    @nomeProduto.setter
    def nomeProduto(self, valor):
        self.__nomeProduto = valor
```

```
# PROPERTY descricaoProduto
@property
def descricaoProduto(self):
    return self.__descricaoProduto

@descricaoProduto.setter
def descricaoProduto(self, valor):
    self.__descricaoProduto = valor
```

Os atributos são criados com a anotação @property



ENCAPSULAMENTO

EXEMPLO PROPRIEDADES CLASSE FORNECEDORES

```
# Método cadastrar
def cadastrarFornecedor(self):

    print("\n=== Cadastro de Fornecedor ===")

    self.nomeFornecedor = input("Digite o nome do fornecedor: ")
    self.nomeProduto = input("Digite o nome do produto: ")
    self.descricaoProduto = input("Digite a descrição do produto: ")

    print("Fornecedor cadastrado com sucesso!")

# Método listar
def listarFornecedor(self):

    print("\n=== Dados do Fornecedor ===")

    print("Fornecedor:", self.nomeFornecedor)
    print("Produto:", self.nomeProduto)
    print("Descrição:", self.descricaoProduto)
```

Ao utilizar properties , o encapsulamento dos métodos e visto como se fosse um método, assim so colocar o nome do construtor self ,seguido de ponto e nome do atributo (. nome do atributo)



ENCAPSULAMENTO

EXEMPLO PROPRIEDADES CLASSE FORNECEDORES

```
from fornecedor import Fornecedores

class Principal:

    @staticmethod
    def main():
        fornecedor = Fornecedores()
        fornecedor.cadastrarFornecedor()
        fornecedor.listarFornecedor()

# equivalente ao main de Java
if __name__ == "__main__":
    Principal.main()
```

Cria o objeto fornecedor e chama os métodos da classe Fornecedores

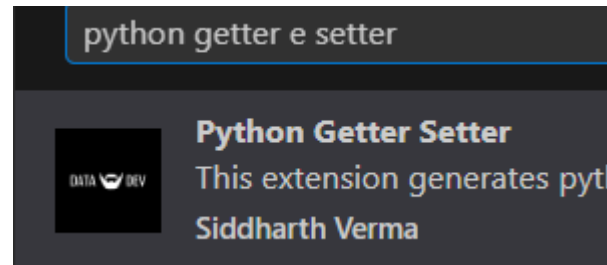


ENCAPSULAMENTO UTILIZANDO PROPRIEDADES – CRIANDO ENCAPSULAMENTO DE FORMA AUTOMÁTICA COM TODOS OS ATRIBUTOS.

Pode criar snippets de código que geram automaticamente @property e setter

Para gerar métodos get e set automaticamente:

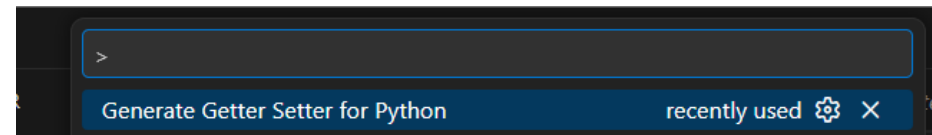
1.Instale a extensão: Vá até a aba de extensões (Ctrl+Shift+X), pesquise por "**Python Getter Setter**" (por Siddharth Verma) e instale.



Digite python getter e setter e instale a extensão

2.Selecione o código: No seu arquivo .py, selecione a variável ou as variáveis (atributos) dentro da classe que você deseja encapsular.

3.Abra a Paleta de Comandos: Pressione Ctrl+Shift+P.



4.Execute o comando: Digite "Generate Getter Setter for Python" e pressione Enter.

5.Resultado: A extensão gerará automaticamente os métodos ou decoradores @property para o atributo selecionado



ENCAPSULAMENTO UTILIZANDO PROPRIEDADES – CRIANDO ENCAPSULAMENTO DE FORMA AUTOMATICA COM TODOS OS ATRIBUTOS.

```
class Fornecedores:

    def __init__(self):
        self.__nomeFornecedor = ""
        self.__nomeProduto = ""
        self.__descricaoProduto = ""

    @property
    def _nomeFornecedor(self):
        return self.__nomeFornecedor

    @_nomeFornecedor.setter
    def _nomeFornecedor(self, value):
        self.__nomeFornecedor = value

    @property
    def _nomeProduto(self):
        return self.__nomeProduto

    @_nomeProduto.setter
    def _nomeProduto(self, value):
        self.__nomeProduto = value

    @property
    def _descricaoProduto(self):
        return self.__descricaoProduto

    @_descricaoProduto.setter
    def _descricaoProduto(self, value):
        self.__descricaoProduto = value
```

Os getter e setters são criados automaticamente após selecionar os atributos e executar o Generate Getter e Setter



ENCAPSULAMENTO

EXEMPLO PROPRIEDADES CLASSE FORNECEDORES

```
# Método cadastrar
def cadastrarFornecedor(self):

    print("\n=== Cadastro de Fornecedor ===")

    self.__nomeFornecedor = input("Digite o nome do fornecedor: ")
    self.__nomeProduto = input("Digite o nome do produto: ")
    self.__descricaoProduto = input("Digite a descrição do produto: ")

    print("Fornecedor cadastrado com sucesso!")

# Método listar
def listarFornecedor(self):

    print("\n=== Dados do Fornecedor ===")

    print("Fornecedor:", self.__nomeFornecedor)
    print("Produto:", self.__nomeProduto)
    print("Descrição:", self.__descricaoProduto)
```

Neste caso ao utilizar properties , o encapsulamento dos atributos é só colocar os atributos como private o nome do construtor self ,seguido de ponto e __nome do atributo (.__nome do atributo)





OBRIGADO

LUIZ.BARRETO@CPS.SP.GOV.BR